

FocusBuddy

Complete Feature Documentation

Phases 1 through 4

Shame-free productivity coaching app for users with executive dysfunction

Stack	React Native (Expo SDK 55), Expo Router v3, NativeWind v4, Supabase v2, Zustand v5, MMKV, OpenAI GPT-4o-mini
Date	March 15, 2026

Table of Contents

Table of Contents

1.1 Root Layout & Auth Management

1.2 Sign In

1.3 Sign Up

1.4 Forgot Password

1.5 Reset Password

1.6 Goal Setup (Onboarding Step 1)

1.7 Notification Time (Onboarding Step 2)

1.8 Trial Confirmation (Onboarding Step 3)

1.9 Tab Navigation

1.10 Home Dashboard

1.11 Focus Timer

1.12 Session Rating

1.13 Daily Check-In

1.14 AI Chat

1.15 Progress Analytics

1.16 Settings

1.17 Shame Emergency

1.18 Bad Day Toolbox

1.19 Paywall

2.1 Database Types

2.2 Supabase Client

- 2.3 MMKV Local Storage**
- 2.4 Auth Store (Zustand)**
- 2.5 useTimer Hook**
- 2.6 useAppState Hook**
- 2.7 Time Utilities**
- 2.8 Constants**
- 2.9 Auth Service**
- 2.10 Check-In Service**
- 2.11 Sessions Service**
- 2.12 Tasks Service**
- 2.13 Chat Service**
- 2.14 Promises Service**
- 2.15 Profile Service**
- 2.16 Goals Service**
- 2.17 Shields Service**
- 2.18 Subscription Service**
- 2.19 AI Chat Edge Function**
- 2.20 AI Check-In Edge Function**
- 2.21 UI Components Library**
- 2.22 Home Components**
- 2.23 Chat Components**
- 2.24 Timer Components**
- 2.25 Check-In Components**
- 2.26 Shared Components**
- 3.1 Progress Screen - Real Data**

3.2 Chat Screen - Real Data

3.3 Settings Screen - Real Data

3.4 Delete Account

3.5 Data Export

4.1 Promises Screen

4.2 Shame Emergency - Real Evidence

4.3 Edit Profile

4.4 QuickActions - Promises Navigation Fix

Project: FocusBuddy - Shame-free productivity coaching app for users with executive dysfunction

Stack: React Native (Expo SDK 55), Expo Router v3, NativeWind v4, Supabase v2, Zustand v5, MMKV, OpenAI GPT-4o-mini

Date: March 15, 2026

1.1 Root Layout & Auth Management

File: `app/_layout.tsx`

Purpose: Root layout managing authentication state, session persistence, route protection, theming, and navigation structure for the entire app.

Functionality:

- Checks existing session on launch via `supabase.auth.getSession()`
- Subscribes to `supabase.auth.onAuthStateChange()` for live auth events
- Implements 30-day inactivity auto-logout via MMKV timestamps
- Protects routes: unauthenticated users forced to auth group; authenticated users redirected away from auth
- Handles PASSWORD_RECOVERY deep link events (navigates to reset-password)
- Loads SpaceMono font, manages splash screen visibility
- Provides light/dark theme via `@react-navigation/native` ThemeProvider
- Manages AppState foreground/background for auto-refresh tokens

How It Works:

1. On mount: fetches session, checks 30-day inactivity timeout
2. `useProtectedRoute()` hook watches route segments + session state
3. If `!session && !inAuthGroup` → redirect to `/(auth)/sign-in`
4. If `session && inAuthGroup` → redirect to `/(tabs)`
5. PASSWORD_RECOVERY event from email link → navigate to `/(auth)/reset-password`
6. AppState listener: starts/stops `supabase.auth.autoRefresh` on foreground/background

Key Details:

- `THIRTY_DAYS_MS = 30 24 60 60 1000`
- Light theme primary: `#8B5CF6`, background: `#FAF5FF`
- Dark theme primary: `#8B5CF6`, background: `#1F1B2E`
- 12 stack screens defined: auth group, onboarding group, tabs group, check-in (fullScreenModal), timer (fullScreenModal, gesture disabled), session-rating (modal, gesture disabled), shame-emergency (fullScreenModal, gesture disabled), bad-day-toolbox, promises, paywall (modal), edit-profile, data-export, delete-account

Dependencies: `expo-router` (Stack, useRouter, useSegments), `expo-font`, `expo-splash-screen`, `@/src/lib/supabase`, `@/src/lib/mmkv` (getLastActivity, setLastActivity, clearLastActivity), `@supabase/supabase-js` (Session type)

1.2 Sign In

File: `app/(auth)/sign-in.tsx`

Purpose: Authenticates existing users by email and password.

Functionality:

- Branded header with "FocusBuddy" logo and tagline "Your shame-free focus companion"
- Email and password inputs with validation
- "Forgot Password?" link, error banner, loading spinner
- Link to Sign Up for new users

How It Works:

1. User enters email (auto-trimmed, case-insensitive) and password
2. `handleSignIn()` validates both non-empty, calls `supabase.auth.signInWithPassword()`
3. On success: route protection redirects to `/(tabs)`
4. On failure: Supabase error message displayed in red banner

Key Details:

- State: `email`, `password`, `loading`, `error`
- Button disabled during loading with opacity change
- `KeyboardAvoidingView` with platform-specific behavior (iOS: padding, Android: height)
- Accessibility labels on all inputs

Navigation: Success → `/(tabs)` | Forgot Password → `/(auth)/forgot-password` | Sign Up → `/(auth)/sign-up`

1.3 Sign Up

File: `app/(auth)/sign-up.tsx`

Purpose: Creates new user accounts and initiates onboarding.

Functionality:

- Header: "Create Account" with tagline "Start your shame-free journey"
- Four fields: display name, email, password, confirm password
- Password hint: "Must be at least 8 characters"

How It Works:

1. Validates: display name non-empty, email non-empty, password 8+ chars, passwords match
2. Calls `supabase.auth.signUp()` with email, password, and `options.data.display_name`

3. On success: navigates to `/(onboarding)/goal-setup`

Key Details:

- Validation stops on first error with descriptive message
- Display name stored in Supabase auth metadata
- State: `displayName`, `email`, `password`, `confirmPassword`, `loading`, `error`

Navigation: Success → `/(onboarding)/goal-setup` | Sign In → `/(auth)/sign-in`

1.4 Forgot Password

File: `app/(auth)/forgot-password.tsx`

Purpose: Sends password reset email with deep link.

Functionality:

- Single email input, "Send Reset Link" button
- Two UI states: email form and green success confirmation

How It Works:

1. Validates email non-empty
2. Calls `supabase.auth.resetPasswordForEmail()` with redirect URL `focusbuddy://reset-password`
3. On success: shows confirmation with email address displayed

Key Details:

- State: `email`, `loading`, `error`, `success`
- Deep link `focusbuddy://reset-password` triggers `PASSWORD_RECOVERY` event in root layout

Navigation: Back to Sign In → `/(auth)/sign-in`

1.5 Reset Password

File: `app/(auth)/reset-password.tsx`

Purpose: Sets new password after clicking email reset link. Only accessible via `PASSWORD_RECOVERY` event.

Functionality:

- New password and confirm password inputs

- "Update Password" button with validation

How It Works:

1. User arrives via deep link → `_layout.tsx` navigates here on `PASSWORD_RECOVERY` event
2. Validates: password non-empty, 8+ chars, passwords match
3. Calls `supabase.auth.updateUser({ password })`
4. On success: Alert modal → "OK" navigates to `/(tabs)` via `router.replace()`

Key Details:

- State: `password`, `confirmPassword`, `loading`, `error`
- Uses `router.replace()` to prevent back navigation to reset screen

Navigation: Success → `/(tabs)` (replace, non-reversible)

1.6 Goal Setup (Onboarding Step 1)

File: `app/(onboarding)/goal-setup.tsx`

Purpose: Collects user's main focus goal (Step 1 of 3).

Functionality:

- Progress bar (1/3 filled), "Step 1 of 3" label
- Multiline text input with 10-500 character validation
- Character counter with color coding (gray < 10, green 10-500, red > 500)
- 4 example goal chips: "Write my thesis", "Launch a side project", "Study for exams", "Build a habit"
- "Continue" button disabled until valid

How It Works:

1. User types goal or taps example chip (sets full text)
2. Character count enforced: `MIN_CHARS=10`, `MAX_CHARS=500`
3. "Continue" passes goal as route param to notification-time

Key Details:

- State: `goal` (string)
- Selected chip shows `bg-primary/10 border-primary` styling
- No external service calls (local state only)

Navigation: Continue → `/(onboarding)/notification-time` with `{ goal }` param

1.7 Notification Time (Onboarding Step 2)

File: `app/(onboarding)/notification-time.tsx`

Purpose: Collects preferred daily check-in notification time (Step 2 of 3).

Functionality:

- Progress bar (2/3 filled), "Step 2 of 3" label
- 4 preset time cards: Morning (8 AM), Midday (12 PM), Afternoon (3 PM), Evening (7 PM)
- "Custom Time" card with 24-hour grid picker (all 24 hours)
- "Continue" button disabled until selection made

How It Works:

1. User taps preset or custom → selects hour
2. Custom shows full 24-hour grid where user picks specific hour
3. "Continue" passes goal + notificationHour as route params

Key Details:

- State: `selectedPreset`, `isCustom`, `customHour`, `showCustomPicker`
- Time display: 12-hour format with AM/PM
- notificationHour converted to string for route params

Navigation: Continue → `/(onboarding)/trial-confirmation` with `{ goal, notificationHour }` params

1.8 Trial Confirmation (Onboarding Step 3)

File: `app/(onboarding)/trial-confirmation.tsx`

Purpose: Confirms trial period and completes onboarding (Step 3 of 3).

Functionality:

- Progress bar (3/3 filled), "You're all set!" header
- Purple gradient illustration placeholder
- Trial info card: 7-day free trial, start/end dates, 3 feature checkmarks
- "Start Using FocusBuddy" green button

How It Works:

1. Displays trial dates (today + 7 days)
2. On button press:
 - Gets user via `supabase.auth.getUser()`

- Calls `completeOnboarding(userId, goal, notificationHour)` (creates goal + sets onboarding_done)
- Inserts subscription record with status "trial", trial_ends_at = +7 days
- Navigates to `/(tabs)` via `router.replace()`

Key Details:

- State: `loading` (boolean)
- Subscription record: entitlement "pro", product_id "trial", period "trial", status "trial"
- Date formatting: `toLocaleDateString('en-US', { month: 'short', day: 'numeric', year: 'numeric' })`

Dependencies: `completeOnboarding` from profile.service, `supabase` for subscription insert

Navigation: Success → `/(tabs)` (replace)

1.9 Tab Navigation

File: `app/(tabs)/_layout.tsx`

Purpose: Bottom tab bar with 5 tabs and floating action button.

Functionality:

- 5 tabs: Home (H), Progress (P), Session (+), Chat (C), Settings (S)
- Floating purple circular "+" button in center for starting sessions
- Session tab intercepts press → navigates to `/timer` instead of showing a tab screen
- Hidden "two" tab (legacy, `href: null`)

Key Details:

- Tab bar: white, height 88px, rounded top corners (24px), absolute positioning
- Active tint: `#8B5CF6` (purple), Inactive: `#9CA3AF` (gray)
- Session button: 56x56 circle, `#8B5CF6` background, shadow, marginBottom 24
- `tabPress` listener on session tab calls `e.preventDefault()` then `router.push('/timer')`

Dependencies: `expo-router` (Tabs, useRouter), React Native (View, Text, Pressable)

1.10 Home Dashboard

File: `app/(tabs)/index.tsx`

Purpose: Primary landing screen showing daily overview, goals, stats, quick actions, and activity feed.

Functionality:

- GreetingHeader with user name and streak count
- Conditional: check-in prompt card OR GoalCard with task list
- StatCards: focus minutes today, current streak
- QuickActions: Start Session, Promises, Chat, Progress
- ActivityFeed: recent sessions and check-ins
- Pull-to-refresh, error banner with retry

How It Works:

1. On mount + focus: fetches `getTodayCheckIn()`, `getRecentSessions(20)`, `getTodayTasks()` in parallel
2. If no check-in: shows "Ready to check in?" card → navigates to `/check-in`
3. If checked in: shows GoalCard with toggleable tasks
4. Task toggle: optimistic UI update, calls `toggleTaskComplete()`, reverts on failure
5. Activity feed built from sessions + check-in data, sorted by timestamp

Key Details:

- State: `refreshing`, `loading`, `error`, `checkIn`, `tasks`, `focusMinutesToday`, `activities`
- `calcTodayFocusMinutes()`: sums `actual_secs` from completed sessions started today
- `buildActivityFeed()`: creates activity array from sessions and check-in
- `useFocusEffect()` for screen-focus refresh

Dependencies: Services: `getTodayCheckIn`, `getTodayTasks`, `toggleTaskComplete`, `getRecentSessions`. Components: `GreetingHeader`, `GoalCard`, `StatCards`, `QuickActions`, `ActivityFeed`

Navigation: To `/timer`, `/(tabs)/chat`, `/(tabs)/progress`, `/promises`, `/check-in`

1.11 Focus Timer

File: `app/timer.tsx`

Purpose: Focus session timer with pause/resume, MMKV persistence, and background reconciliation.

Functionality:

- **Setup mode (no params):** Task name input, duration presets (25/50/Custom), Start button
- **Running mode:** Large elapsed time display, planned duration, pause/resume button, end session button, "Goal Reached" banner when elapsed $\geq$ planned

How It Works:

1. Entry from check-in: receives `taskName`, `plannedMinutes`, `taskId`, `checkInId` → skips setup
2. Entry from quick action: no params → shows setup screen

3. Timer state persisted to MMKV keys: `startedAt`, `pausedAt`, `accumulated`, `isPaused`, `taskName`, `plannedMinutes`
4. On mount: checks MMKV for existing timer (restores if found)
5. 1-second interval recalculates elapsed: `accumulated + (now - startedAt) / 1000`
6. AppState listener: on foreground, recalculates elapsed to prevent drift
7. Pause: stores accumulated, sets `isPaused` flag
8. Resume: resets `startedAt` to `now`, clears pause flag
9. End: calculates final duration, clears MMKV, navigates to session-rating

Key Details:

- State: `setupTaskName`, `setupMinutes` (5-180), `isCustomSetup`, `elapsedSeconds`, `isPaused`, `goalReached`
- Custom duration validated: 5-180 minute range, numeric-only input
- MMKV keys: `timer_started_at`, `timer_paused_at`, `timer_accumulated`, `timer_is_paused`, `timer_task_name`, `timer_planned_minutes`

Dependencies: `storage` from MMKV, Components: `TimerDisplay`, `TimerControls`

Navigation: End → `/session-rating` via `router.replace()` with `{ taskName, durationSeconds, plannedMinutes, taskId, checkInId }`

1.12 Session Rating

File: `app/session-rating.tsx`

Purpose: Post-session feedback with 4-point rating, optional notes, and struggle support.

Functionality:

- Session summary card: task name, actual/planned duration
- 4 rating cards: Great (green), Good (blue), OK (yellow), Struggled (orange)
- Optional note input (500 char max with counter)
- "Want to talk about it?" banner (appears only if Struggled selected)
- Save button with loading state

How It Works:

1. Receives params: `taskName`, `durationSeconds`, `plannedMinutes`, `taskId`, `checkInId`
2. User selects rating → card highlights with scale transform (1.03)
3. If Struggled: shows chat link with prefill message
4. On save:

- Maps rating to numeric: great=4, good=3, ok=2, struggled=1
- `createSession()` → `endSessionService()` (with rating, note)
- Fire-and-forget: `toggleTaskComplete(taskId, true), supabase.rpc('update_streak')`

5. Navigates home via `router.dismissAll()` or `router.replace('/')`

Key Details:

- State: `selectedRating`, `note`, `isSaving`
- Prefill for chat: "I just finished a X minute session on 'Y' and I'm struggling..."
- `formatDuration()`: formats seconds as "XhYm", "Xm Ys", or "Xs"

Dependencies: **Services:** `createSession`, `endSessionService`, `toggleTaskComplete`,
`supabase.rpc`

Navigation: Save → home | Chat → `/(tabs)/chat` with prefill

1.13 Daily Check-In

File: `app/check-in.tsx`

Purpose: Multi-step daily questionnaire with AI-powered task planning.

Functionality:

- **Step 1 - Energy Selection:** 4 buttons (Good=5, Meh=3, Low=1, Need-a-Break=0)
- **Step 2 - AI Conversation:** User types goal, AI responds with assessment, handles clarification for vague goals
- **Step 3 - Task Review:** AI-generated task list, toggle in/out, "Make Smaller" (halves time, downgrades difficulty), "Try different approach" (regenerate), time commitment selector (15/25/50/Custom min), "Let's do this" button
- **Step 5 - Need a Break:** Bad Day Toolbox link, rest day with streak shield, go back
- **Already Checked In:** Success state if check-in exists for today

How It Works:

1. On mount: `getTodayCheckIn()` checks for existing check-in
2. Energy selection → if "need-a-break" → Step 5; otherwise → Step 2
3. AI call: `supabase.functions.invoke('ai-checkin')` with `energyLevel` + message
4. If response has `clarification` field: shows clarification UI, re-submits
5. Task mapping: `{ id, title, estimatedMinutes, difficulty, included: true }`
6. "Make Smaller": halves `estimatedMinutes`, downgrades difficulty (hard→medium→easy)
7. "Regenerate": calls AI again with "suggest a DIFFERENT approach" instruction
8. Confirmation: marks check-in completed, navigates to `/timer` with first included task

Key Details:

- Steps: 1, 2, 3, 5 (no step 4)
- `energyToNumber()`: maps UI enum to numeric value
- State: `selectedEnergy`, `userMessage`, `tasks`, `timeCommitment`, `aiMessage`, `aiLoading`, `checkInId`, `clarificationQuestion`, `alreadyCheckedIn`
- ProgressDots sub-component: 3 dots indicating current step
- Rest day shield: decrements `streak_shields` via Supabase update, refreshes profile

Dependencies: Services: `getTodayCheckIn`, `supabase.functions.invoke('ai-checkin')`.
Components: `EnergySelector`, `TaskBreakdown`

Navigation: Confirm → `/timer` | Bad Day Toolbox → `/bad-day-toolbox` | Chat link → `/(tabs)/chat`

1.14 AI Chat

File: `app/(tabs)/chat.tsx`

Purpose: Real-time AI companion chat with message history, rate limiting, shame detection, and quick actions.

Functionality:

- Inverted FlatList: newest messages at bottom, user right-aligned, AI left-aligned
- Typing indicator while AI processes
- Multiline input with send button, quick action buttons above
- Welcome message if no history
- Prefill support from other screens (e.g., session rating)
- Pagination: loads 20 messages per page, infinite scroll at 30% threshold
- Rate limit toast (auto-dismisses after 5s)
- Shame detection: auto-navigates to `/shame-emergency`
- Long-press message deletion with confirmation

How It Works:

1. On mount: loads `getMessages(userId, 20, 0)`, filters to user/assistant roles
2. Send: optimistic user message → `supabase.functions.invoke('ai-chat')` → reload from DB
3. Rate limit: checks `data?.error === 'rate_limited'`, shows toast, removes optimistic message
4. Shame: checks `data?.shame_detected`, navigates to shame-emergency
5. Pagination: `handleLoadMore()` at 30% threshold, appends older messages
6. Delete: removes locally, calls `deleteMessage(id)` fire-and-forget

Key Details:

- `PAGE_SIZE = 20`
- State: `messages`, `inputText`, `isTyping`, `isLoading`, `isLoadingMore`, `hasMore`, `rateLimitMessage`
- Refs: `flatListRef`, `prefillApplied`, `offsetRef`
- Cancellation token in `useEffect` prevents race conditions

Dependencies: Services: `getMessages`, `deleteMessage`, `supabase.functions.invoke('ai-chat')`.
Components: `MessageBubble`, `QuickActions` (chat), `TypingIndicator`

Navigation: Shame detected → `/shame-emergency`

1.15 Progress Analytics

File: `app/(tabs)/progress.tsx`

Purpose: Analytics dashboard with charts, milestones, insights, and session history.

Functionality:

- Period selector: This Week / This Month / All Time
- Stat cards: Total Focus (hours), Sessions (count), Avg Duration (minutes) with trend indicators
- Focus Time bar chart: 7-day view (Mon-Sun) with average line
- Session Breakdown: horizontal bars by rating (Great/Good/OK/Struggled)
- Insights: best day, avg session, total sessions, total hours
- Milestones: First Session, 10/25/100 Sessions, 10/50 Hours, 7/30-day Streak
- Recent Sessions list with task name, date, duration, rating badge
- Loading state, empty state, pull-to-refresh

How It Works:

1. Fetches `getRecentSessions(200)`, `getSessionStats()`, milestones query in parallel
2. `useMemo` computations: filtered sessions, weekly chart, rating distribution, insights, milestones
3. Trend calculation: compares this week vs last week
4. Milestone status: earned (filled), in-progress (partial), locked (empty)

Key Details:

- Types: `Period`, `ChartDay`, `RatingRow`, `MilestoneDisplay`
- State: `selectedPeriod`, `refreshing`, `loading`, `allSessions`, `stats`, `earnedMilestones`
- 10 sub-components: `PeriodSelector`, `StatCard`, `FocusChart`, `SessionBreakdown`, `InsightsSection`, `MilestonesSection`, `RatingBadge`, `SessionHistoryItem`, `EmptyState`, `LoadingState`

Dependencies: Services: `getRecentSessions`, `getSessionStats`, `supabase.from('milestones')`

1.16 Settings

File: `app/(tabs)/settings.tsx`

Purpose: User settings hub for profile, preferences, privacy, subscription, and account management.

Functionality:

- **Profile:** Avatar (initials), display name, email, "Edit Profile" button
- **Preferences:** Notification Time, Default Session Length (15/25/50/Custom), Celebration Style (Enthusiastic/Moderate/Minimal), Theme (Light/Dark/Auto)
- **Privacy:** Anonymous Sharing toggle, Analytics toggle
- **Subscription:** Current plan badge, Upgrade to Pro button
- **Data:** Export My Data, Delete Account (red danger)
- **Other:** Restore Purchases
- **Sign Out:** Confirmation alert → `signOut()`
- **Footer:** Version "FocusBuddy v1.0.0 (Build 1)"

How It Works:

- Profile data from auth store: `profile?.display_name`, `user?.email`
- Preference changes: local state update + `updateProfile()` call (optimistic, silent fail)
- `DropDownPicker` generic component for selections
- Sign out: Alert confirmation → `useAuthStore.getState().signOut()`

Key Details:

- State: `sessionLength`, `celebrationStyle`, `theme`, `anonymousSharing`, `analyticsEnabled`, `isSubscribed` (hardcoded false)
- Sub-components: `SectionCard`, `SettingRow`, `ToggleRow`, `DropDownPicker<T>`

Navigation: Edit Profile → `/edit-profile` | Upgrade → `/paywall` | Export → `/data-export` | Delete → `/delete-account`

1.17 Shame Emergency

File: `app/shame-emergency.tsx`

Purpose: Crisis support for users experiencing productivity shame. Multi-phase intervention with facts, evidence, and micro-actions.

Functionality:

- **Phase 1 (STOP):** Animated "STOP" text (800ms fade), subtitle (1s delay), auto-advances to Choose after 3.5s
- **Phase 2 (Choose):** 4 paths: Read the Facts, Show Me Evidence, I Need to Talk, Tiny Steps

- **Phase 3 (Facts):** 7 science-backed shame facts with explanations
- **Phase 4 (Evidence):** Real stats (sessions completed, promises kept, streak, promise rate) with personalized encouragement
- **Phase 5 (Tiny-Steps):** 4 micro-actions (stretch, water, open file, 2-min work) with selection confirmation

How It Works:

- Phase state machine: `'stop' | 'choose' | 'facts' | 'evidence' | 'tiny-steps'`
- Animations: `Animated.timing` for fade-in effects
- Evidence lazy-loaded on first entry to phase 4
- Evidence fetches `getSessionStats()` + `getPromiseHistory(30)` in parallel
- Calculates promise rate: `(kept / resolved) * 100`
- Dynamic encouragement based on actual stats vs. fallback for new users

Key Details:

- State: `phase`, `showBackButton`, `selectedMicroAction`, `evidenceStats`, `evidenceLoading`
- 7 SHAME_FACTS with title/body pairs
- 4 MICRO_ACTIONS with label/description
- LinearGradient background: `['#E0E7FF', '#EDE9FE', '#F5F3FF']`

Dependencies: Services: `getSessionStats`, `getPromiseHistory`, auth store for `profile.current_streak`

Navigation: "I Need to Talk" → `/(tabs)/chat` | Close → `router.back()`

1.18 Bad Day Toolbox

File: `app/bad-day-toolbox.tsx`

Purpose: Categorized self-care activities for low-energy days with difficulty levels and fallback suggestions.

Functionality:

- 4 category tabs: Physical, Work, Comfort, Tomorrow (4-5 actions each)
- Action cards: title, description, difficulty badge (Very Easy/Easy/Medium), time estimate
- Commitment flow: "I'll Do This" → "Done" / "Couldn't Do It"
- Fallback on "Couldn't Do It": simpler suggestion per category (e.g., "taking three deep breaths")

How It Works:

1. Category selection resets committed state
2. "I'll Do This" → card highlights, shows Done/Couldn't buttons
3. "Done" → `router.back()`

4. "Couldn't Do It" → shows fallback message with "Show me other options" reset

Key Details:

- State: `selectedCategory`, `committedActionIndex`, `commitState` ('idle'|'committed'|'couldnt')
- `DIFFICULTY_COLORS`: Very Easy (green), Easy (blue), Medium (yellow)
- `SIMPLER_FALLBACKS`: Physical="deep breaths", Work="opening one app", Comfort="blanket", Tomorrow="one alarm"

Dependencies: Constants: `BAD_DAY_CATEGORIES`, `DEFAULT_BAD_DAY_ACTIONS`

Navigation: Done → `router.back()` | Stay on screen for reset/retry

1.19 Paywall

File: `app/paywall.tsx`

Purpose: Subscription screen with pricing, feature list, and trial/restore buttons.

Functionality:

- Purple "FB" logo, "Unlock FocusBuddy Pro" header
- 4 features: Unlimited AI chat, Advanced analytics, Custom timer presets, Priority support
- Plan cards: Monthly (\$20/mo), Annual (\$180/yr with "Save \$60" badge)
- "Start Free Trial" button (green), "Restore Purchase" link
- Close button only visible if `trialActive=true` param

Key Details:

- State: `selectedPlan` ('monthly'|'annual', defaults 'annual')
- **Note:** RevenueCat integration not yet implemented. Button handlers are placeholders.

Navigation: Close/Trial → `router.back()`

2.1 Database Types

File: `src/types/database.ts`

Purpose: Supabase-compatible TypeScript type definitions. Uses `type` aliases (not `interface`) for postgres-js GenericTable compatibility.

Exported Types:

- **Json** — Flexible JSON union type
- **Profile** — 12 fields: `id`, `display_name`, `onboarding_done`, `notification_hour`, `celebration_style`, `theme`, `streak_shields` (max 3), `current_streak`, `longest_streak`, `trust_score` (0-100), `created_at`, `updated_at`
- **Goal** — 7 fields: `id`, `user_id`, `title`, `description`, `is_active`, `created_at`, `archived_at`
- **DailyCheckIn** — 10 fields: `id`, `user_id`, `goal_id`, `check_in_date`, `energy_level` (1-5), `mood_note`, `today_goal_text`, `completed`, `ai_messages` (JSON), `created_at`
- **Task** — 9 fields: `id`, `user_id`, `check_in_id`, `title`, `estimated_mins`, `order_index`, `completed`, `completed_at`, `created_at`
- **FocusSession** — 14 fields: `id`, `user_id`, `task_id`, `check_in_id`, `started_at`, `ended_at`, `planned_mins`, `actual_secs`, `pauses`, `rating` (1-4), `rating_label`, `note`, `status` ('active'|'paused'|'completed'|'abandoned'), `created_at`
- **ChatMessage** — 7 fields: `id`, `user_id`, `role` ('user'|'assistant'|'system'), `content`, `context`, `tokens_used`, `created_at`
- **PromiseRecord** — 7 fields: `id`, `user_id`, `promise_date`, `text`, `kept` (null=pending, true=kept, false=broken), `kept_at`, `created_at`
- **AiRateLimit** — 4 fields: `id`, `user_id`, `window_start`, `message_count`
- **Milestone** — 5 fields: `id`, `user_id`, `type`, `achieved_at`, `seen`
- **Subscription** — 9 fields: `id`, `user_id`, `revenuecat_user_id`, `entitlement`, `product_id`, `period`, `trial_ends_at`, `current_period_ends`, `status` ('active'|'expired'|'trial'|'cancelled'), `updated_at`
- **BadDayAction** — 5 fields: `id`, `user_id`, `category`, `action_text`, `completed_at`
- **Database** — Interface with Row/Insert/Update types for all tables

2.2 Supabase Client

File: `src/lib/supabase.ts`

Purpose: Initializes typed Supabase client with secure token storage.

Exports: `supabase` — `SupabaseClient<Database>` with `ExpoSecureStoreAdapter`, auto-refresh, session persistence enabled.

Environment Variables: `EXPO_PUBLIC_SUPABASE_URL`, `EXPO_PUBLIC_SUPABASE_ANON_KEY`

2.3 MMKV Local Storage

File: `src/lib/mmkv.ts`

Purpose: Fast local key-value storage for timer state and activity tracking.

Exports:

- `storage` — MMKV instance with typed get/set/delete methods
 - `TimerState` — Interface: `sessionId`, `startedAt` (ms), `elapsedSecsBeforePause`, `pausedAt` (ms|null), `status` ('running'|'paused'|'stopped')
 - `getTimerState() → TimerState | null` — Parses JSON from storage
 - `setTimerState(state)` — Persists as JSON
 - `clearTimerState()` — Deletes timer state
 - `setLastActivity()` — Records current timestamp for inactivity tracking
 - `getLastActivity() → number | null` — Returns last activity timestamp
 - `clearLastActivity()` — Deletes timestamp on logout
-

2.4 Auth Store (Zustand)

File: `src/stores/authStore.ts`

Purpose: Global authentication state management with Supabase integration.

State: `user` (User|null), `session` (Session|null), `profile` (Profile|null), `isLoading` (boolean), `isAuthenticated` (derived)

Actions:

- `initialize() → { unsubscribe }` — Subscribes to auth state changes, auto-fetches profile
 - `signUp(email, password)` — Creates account via Supabase
 - `signIn(email, password)` — Signs in via Supabase
 - `signOut()` — Signs out, clears profile
 - `resetPassword(email)` — Sends reset email
 - `fetchProfile()` — Fetches profile from DB
 - `updateProfile(updates: Partial<Profile>)` — Updates profile with auto-timestamp
-

2.5 useTimer Hook

File: `src/hooks/useTimer.ts`

Purpose: Persistent focus session timer with background reconciliation.

Returns: `{ elapsedSeconds, isPaused, isRunning, start, pause, resume, stop, restore }`

Key Behavior:

- `start(sessionId)` — Initializes with MMKV persistence, starts 1s interval
 - `pause()` — Records elapsed, increments pause count, stops interval
 - `resume()` — Recalculates `startedAt = now - elapsedBefore * 1000`, resumes interval
 - `stop()` → `{ totalSeconds, pauseCount }` — Clears MMKV, returns stats
 - `restore()` — Loads from MMKV on mount, reconciles background time
 - AppState listener recalculates on foreground to catch background drift
-

2.6 useAppState Hook

File: `src/hooks/useAppState.ts`

Purpose: App lifecycle listener for foreground/background transitions.

Signature: `useAppState(onForeground?, onBackground?)` → `MutableRefObject<AppStateStatus>`

2.7 Time Utilities

File: `src/utils/time.ts`

Exported Functions:

- `formatTimer(totalSeconds)` → "MM:SS" or "HH:MM:SS"
 - `formatDuration(minutes)` → "25 min", "1h", "1h 25m"
 - `getRelativeTime(dateString)` → "Just now", "5m ago", "2h ago", "Yesterday", "3d ago", "Mar 15"
 - `getGreeting()` → "Good morning/afternoon/evening/night"
 - `getTimeOfDay()` → 'morning'|'afternoon'|'evening'|'night'
 - `formatDate(dateString)` → "Mon, Mar 15"
 - `isToday(dateString)` → boolean
 - `getTodayDateString()` → "YYYY-MM-DD"
-

2.8 Constants

File: `src/utils/constants.ts`

Key Constants:

- **App:** `APP_NAME='FocusBuddy'`, `APP_TAGLINE='Your shame-free focus companion'`
 - **Timer:** `DEFAULT_SESSION_MINUTES=25`, `SESSION_DURATIONS=[15, 25, 50]`, range 5-180
 - **Rate Limiting:** `CHAT_RATE_LIMIT=100` msgs/hr, `CHAT_RATE_WINDOW_MS=3600000`
 - **Streaks:** `STREAK_SHIELD_MAX=3`, `STREAK_SHIELD_EARN_DAYS=7`
 - **Trust Score:** Weights: `keptRatio=0.6`, `streak=0.2`, `activity30d=0.2`
 - **Trust Labels:** 0-30 "Rebuilding" (orange), 31-60 "Growing" (amber), 61-80 "Strong" (green), 81-100 "Rock Solid" (purple)
 - **Energy Levels:** 4=Good (green), 3=Meh (amber), 2=Low (orange), 1=Need-a-Break (indigo)
 - **Session Ratings:** 4=Great (green), 3=Good (blue), 2=OK (amber), 1=Struggled (orange)
 - **Celebration Styles:** enthusiastic, moderate, minimal
 - **Bad Day Actions:** 16 actions across 4 categories with difficulty levels
 - **Shame Patterns:** 12 patterns ("i'm lazy", "i'm broken", "i'm a failure", etc.)
 - **Milestones:** 8 types (`first_session`, `streak_7`, `streak_30`, `hours_10`, `hours_50`, `sessions_25`, `sessions_100`, `promises_10`)
-

2.9 Auth Service

File: `src/services/auth.service.ts`

Exported Functions:

- `signup(email, password)` — Creates account + inserts profile with defaults (`trust_score=50`, `notification_hour=9`, `celebration_style='confetti'`, `theme='system'`)
 - `signIn(email, password)` — Email/password authentication
 - `signOut()` — Logout
 - `resetPassword(email)` — Sends reset email
 - `getCurrentSession()` → `Session | null`
 - `onAuthStateChange(callback)` → `Subscription` (unsubscribable)
-

2.10 Check-In Service

File: `src/services/checkin.service.ts`

Exported Functions:

- `getTodayCheckIn(userId) → DailyCheckIn | null` — Filters by today's date
 - `startCheckIn(userId, energyLevel, goalId) → DailyCheckIn` — Invokes `ai-checkin` Edge Function
 - `getRecentCheckIns(userId, days) → DailyCheckIn[]` — Past N days, newest first
-

2.11 Sessions Service

File: `src/services/sessions.service.ts`

Types: `SessionStats` — { totalSessions, totalTimeSecs, avgDurationSecs, ratingDistribution: Record<number, number> }

Exported Functions:

- `createSession(userId, taskId, checkInId, plannedMins) → FocusSession` — Status 'active', actual_secs=0, pauses=0
 - `updateSession(sessionId, updates) → FocusSession` — Partial update
 - `endSession(sessionId, actualSecs, rating, ratingLabel, note?) → FocusSession` — Sets status='completed', ended_at=now
 - `getRecentSessions(userId, limit) → FocusSession[]` — Newest first
 - `getSessionStats(userId) → SessionStats` — Aggregated from completed sessions
-

2.12 Tasks Service

File: `src/services/tasks.service.ts`

Exported Functions:

- `getTasksForCheckIn(checkInId) → Task[]` — Ordered by order_index
 - `getTodayTasks(userId) → Task[]` — Inner join with daily_check_ins, today's date
 - `toggleTaskComplete(taskId, completed) → Task` — Sets completed_at=now or null
 - `createTask(userId, checkInId, title, estimatedMins, orderIndex) → Task`
-

2.13 Chat Service

File: `src/services/chat.service.ts`

Types: `ChatResponse` — { message, tokensUsed }; `RateLimitStatus` — { allowed, remaining, resetAt }

Exported Functions:

- `sendMessage(userId, content, context) → ChatResponse` — Invokes `ai-chat` Edge Function
 - `getMessages(userId, limit, offset) → ChatMessage[]` — Paginated, newest first
 - `deleteMessage(messageId)` — Soft-delete
 - `clearHistory(userId)` — Bulk delete
 - `checkRateLimit(userId) → RateLimitStatus` — 20 messages/day free tier, 24-hour rolling window
-

2.14 Promises Service

File: `src/services/promises.service.ts`

Types: `TrustScore` — { score (0-100), keptRatio, streakBonus, activityBonus }

Exported Functions:

- `getTodayPromise(userId) → PromiseRecord | null`
- `createPromise(userId, text) → PromiseRecord` — kept=null (pending)
- `updatePromise(promiseId, kept) → PromiseRecord` — Sets kept_at=now
- `getPromiseHistory(userId, days) → PromiseRecord[]` — Newest first
- `calculateTrustScore(userId) → TrustScore`

Trust Score Formula:

```
score = (keptRatio * 0.6) + (streakBonus * 0.2) + (activityBonus * 0.2)
```

- **keptRatio** (0-1): kept / resolved (default 0.5 if no resolved)
 - **streakBonus** (0-1): consecutive kept from most recent, capped at 30 days
 - **activityBonus** (0-1): days with promise in last 30, normalized to 30
-

2.15 Profile Service

File: `src/services/profile.service.ts`

Exported Functions:

- `getProfile(userId) → Profile`
-

- `updateProfile(userId, updates) → Profile` — Auto-sets `updated_at`
 - `completeOnboarding(userId, goalTitle, notificationHour)` — Creates goal + sets `onboarding_done=true`
 - `deleteAccount(userId)` — Soft-delete + `signOut`
-

2.16 Goals Service

File: `src/services/goals.service.ts`

Exported Functions:

- `getActiveGoal(userId) → Goal | null` — `is_active=true`, `archived_at=null`, newest first
 - `createGoal(userId, title, description?) → Goal` — Deactivates existing goals first
 - `archiveGoal(goalId) → Goal` — Sets `is_active=false`, `archived_at=now`
 - `getGoals(userId, includeArchived?) → Goal[]` — Optional archived filter
-

2.17 Shields Service

File: `src/services/shields.service.ts`

Exported Functions:

- `checkAndAwardShield(userId, currentStreak, currentShields) → number` — Awards shield if `streak % 7 === 0` and `shields < 3`
- `useShield(userId, currentShields) → boolean` — Consumes shield, returns success
- `getShieldCount(userId) → number`

Constants: `SHIELD_MAX=3`, `SHIELD_EARN_DAYS=7`

2.18 Subscription Service

File: `src/services/subscription.service.ts`

Types: `TrialStatus` — `{ isTrialing, daysLeft, trialEndsAt }`

Exported Functions:

- `getSubscription(userId) → Subscription | null`
 - `updateSubscription(userId, updates) → Subscription` — Auto-sets `updated_at`
 - `checkTrialStatus(userId) → TrialStatus` — Checks `status='trial'` + `trial_ends_at`
-

2.19 AI Chat Edge Function

File: `supabase/functions/ai-chat/index.ts`

Purpose: Deno Edge Function for shame-aware AI chat with context building and rate limiting.

Features:

- **Authentication:** Validates JWT, creates user-scoped and service-scoped Supabase clients
- **Rate Limiting:** 100 messages/hour rolling window via `ai_rate_limits` table
- **Shame Detection:** 19 patterns (case-insensitive): "i'm lazy", "i'm broken", "i'm a failure", "i'm worthless", "what's wrong with me", etc.
- **Context Building:** Fetches profile, active goal, today's check-in, today's promise, recent 3 sessions, trust score, celebration style
- **System Prompt Forbidden Patterns:** "you should", "you haven't", "you need to", "just" (minimizing), comparative language, "why didn't you", unsolicited streak references, laziness/failure implications
- **OpenAI:** Model `gpt-4o-mini`, `max_tokens=2000`, `temperature=0.7`
- **Persistence:** Saves both user and assistant messages to `chat_messages` table with context (`shame_detected`, `goal_id`)

Request: `{ message: string }`

Response: `{ content: string, shame_detected: boolean, tokens_used: number }`

Errors: 401 (auth), 400 (invalid), 429 (rate limited), 502 (OpenAI), 500 (unhandled)

2.20 AI Check-In Edge Function

File: `supabase/functions/ai-checkin/index.ts`

Purpose: Deno Edge Function for energy-aware daily task planning.

Features:

- **Vague Goal Detection:** Goals < 10 chars or single word → clarification response
- **Task Breakdown:** 2-5 tasks scaled to energy level (1-2: easy/short, 4-5: medium/hard options)
- **Context:** Profile, active goal, recent sessions, time-of-day
- **System Prompt:** ADHD-aware, shame-free, forbidden patterns same as ai-chat
- **JSON Parsing:** Handles fenced (````json`) and unfenced responses
- **DB Operations:** Upserts `daily_check_ins`, inserts tasks with `order_index`

Request: `{ energyLevel: number (1-5), message?: string }`

Response (tasks): { message, tasks: [{id, title, estimated_mins, difficulty, order_index, completed}], check_in_id, rationale, tokens_used }

Response (clarification): { clarification: string, tokens_used }

2.21 UI Components Library

Directory: `src/components/ui/`

Component	File	Purpose	Key Props
Button	<code>Button.tsx</code>	Pressable button with variants	title, onPress, variant (primary/secondary/ghost/danger/accent), size (sm/md/lg), loading, icon, fullWidth
Card	<code>Card.tsx</code>	Container with optional header	variant (default/elevated/outlined), onPress?, header?, children
Input	<code>Input.tsx</code>	Text input with label/error	label?, error?, helperText?, secureTextEntry? + all TextInputProps
Avatar	<code>Avatar.tsx</code>	User avatar (image or initials)	name, imageUri?, size (sm/md/lg)
Badge	<code>Badge.tsx</code>	Status indicator	text, variant (default/success/warning/danger/info)
ProgressBar	<code>ProgressBar.tsx</code>	Animated progress bar	progress (0-1), color?, height?
BottomSheet	<code>BottomSheet.tsx</code>	Slide-up modal	visible, onClose, title?, children
LoadingScreen	<code>LoadingScreen.tsx</code>	Full-screen spinner	message?
SafeView	<code>SafeView.tsx</code>	Screen wrapper with safe area	children, className?

Component	File	Purpose	Key Props
EmptyState	EmptyState.tsx	No-content placeholder	icon?, title, description?, actionTitle?, onAction?

2.22 Home Components

Directory: `src/components/home/`

Component	File	Purpose	Key Props
GreetingHeader	GreetingHeader.tsx	Time-based greeting + streak	name, streak
StatCards	StatCards.tsx	Focus minutes + streak cards	focusMinutes, streak
GoalCard	GoalCard.tsx	Goal with task checklist + progress	goalText, tasks[], onToggleTask
QuickActions	QuickActions.tsx	4 action buttons grid	onStartSession, onChat, onProgress, onPromises
ActivityFeed	ActivityFeed.tsx	Recent activity list (5 max)	activities[] (5 types: session/checkin/promise/milestone/streak)

2.23 Chat Components

Directory: `src/components/chat/`

Component	File	Purpose	Key Props
MessageBubble	MessageBubble.tsx	Chat message with delete	message (ChatMessage), onDelete?

Component	File	Purpose	Key Props
QuickActions	QuickActions.tsx	5 preset chat prompts	onSelectAction (scrollable: "I'm stuck", "I'm procrastinating", "Break down task", "I can't start", "Show my progress")

2.24 Timer Components

Directory: `src/components/timer/`

Component	File	Purpose	Key Props
TimerDisplay	TimerDisplay.tsx	Elapsed + planned time display	elapsedSeconds, plannedMinutes, isPaused
TimerControls	TimerControls.tsx	Play/pause + end buttons	isPaused, onPause, onResume, onEnd

2.25 Check-In Components

Directory: `src/components/checkin/`

Component	File	Purpose	Key Props
EnergySelector	EnergySelector.tsx	4 energy level cards (2x2 grid)	selectedEnergy, onSelect
TaskBreakdown	TaskBreakdown.tsx	Task list with toggles + "make smaller"	tasks[], onToggle, onMakeSmaller

2.26 Shared Components

Directory: `src/components/shared/`

Component	File	Purpose
TypingIndicator	<code>TypingIndicator.tsx</code>	3-dot animated indicator (150ms stagger, translateY -6, opacity 0.4-1.0)

Phase 3 replaced mock/stub data with real Supabase queries on 5 screens.

3.1 Progress Screen - Real Data

File: `app/(tabs)/progress.tsx`

What Changed: Replaced mock session arrays with real Supabase queries.

Data Integration:

- `getRecentSessions(userId, 200)` — Fetches up to 200 real sessions
- `getSessionStats(userId)` — Aggregated stats from completed sessions
- `supabase.from('milestones').select('*').eq('user_id', userId)` — Earned milestones

Pattern: `useFocusEffect` → parallel `Promise.all` fetch → `useMemo` derived computations → pull-to-refresh via `RefreshControl`

States: Loading spinner, empty state, error fallback with `console.error`

3.2 Chat Screen - Real Data

File: `app/(tabs)/chat.tsx`

What Changed: Replaced welcome-only UI with real message history and Edge Function AI calls.

Data Integration:

- `getMessages(userId, 20, offset)` — Paginated real chat history
- `supabase.functions.invoke('ai-chat')` — Real AI responses via Edge Function
- `deleteMessage(messageId)` — Real DB deletion

Pattern: `useEffect` initial load → optimistic UI for sends → reload from DB after AI response → pagination via offset tracking → cancellation tokens prevent race conditions

Special Handling: Rate limit detection (`data?.error === 'rate_limited'`), shame detection (`data?.shame_detected`), prefill param support from other screens

3.3 Settings Screen - Real Data

File: `app/(tabs)/settings.tsx`

What Changed: Replaced hardcoded preferences with real profile data persistence.

Data Integration:

- Profile data from `useAuthStore: profile?.display_name, profile?.celebration_style, profile?.theme, profile?.notification_hour`
- Preference updates via `useAuthStore.getState().updateProfile({ ... })`

Pattern: Local state initialized from profile → optimistic local update → async profile update (silent fail)

3.4 Delete Account

File: `app/delete-account.tsx`

Purpose: Account deletion with confirmation.

Data Integration:

- Soft-delete: `supabase.from('profiles').update({ display_name: '[deleted]' }).eq('id', userId)`
- Sign out: `useAuthStore.getState().signOut()`

Key Details:

- Requires typing "DELETE" exactly to enable button
 - State: `confirmText, isDeleting`
 - Shows 30-day recovery window info in success alert
 - Error handling with `Alert.alert`
-

3.5 Data Export

File: `app/data-export.tsx`

Purpose: Exports all user data from 5 tables.

Data Integration:

- Parallel queries via `Promise.all`:
- `supabase.from('focus_sessions').select('*').eq('user_id', userId)`
- `supabase.from('daily_check_ins').select('*').eq('user_id', userId)`
- `supabase.from('tasks').select('*').eq('user_id', userId)`
- `supabase.from('promises').select('*').eq('user_id', userId)`
- `supabase.from('chat_messages').select('*').eq('user_id', userId)`

Key Details:

- Validates no errors in responses
-

- Counts total records across all tables
 - State: `requested`, `isExporting`
 - **Note:** Email delivery of export not yet implemented (Phase 5)
-

Phase 4 replaced the last mock/stub data in 3 screens and fixed a navigation gap.

4.1 Promises Screen

File: `app/promises.tsx`

Purpose: Daily promise system for building self-trust with trust score visualization.

What Changed: Replaced `MOCK_TRUST_SCORE=54` and `MOCK_HISTORY` with real service calls.

Data Integration:

- `getTodayPromise(userId)` — Today's promise or null
- `createPromise(userId, text)` — Creates new promise
- `updatePromise(promiseId, kept)` — Marks as kept/broken
- `getPromiseHistory(userId, 7)` — Past week's promises
- `calculateTrustScore(userId)` — EMA-based trust score

Promise Lifecycle:

1. No promise today → show input with example chips ("I will open my laptop at 10am", etc.)
2. Promise created → show card with "Did you keep your promise?" buttons
3. "Yes" → `updatePromise(id, true)` → green "Promise Kept" badge
4. "No" → reason picker (Too big, Forgot, Life happened, Just couldn't) → `updatePromise(id, false)` → red badge
5. History + trust score refresh after every action

Key Details:

- State: `todayPromise, history, trustScore, showReasonPicker, isLoading, isSaving, refreshing`
 - Trust label: `getTrustLabel(score)` maps to `TRUST_LABELS` tiers
 - Uses `useFocusEffect` + pull-to-refresh
 - Uses `PromiseRecord` type (kept: boolean|null) instead of old string status
-

4.2 Shame Emergency - Real Evidence

File: `app/shame-emergency.tsx`

What Changed: Replaced hardcoded evidence stats (12 sessions, 8 promises, 5-day streak, 73% rate) with real data.

Data Integration:

- `getSessionStats(userId)` → `sessionsCompleted = totalSessions`
- `getPromiseHistory(userId, 30)` → calculates kept count and promise rate
- `profile?.current_streak` → real streak from auth store

Evidence Calculation:

```
const resolved = promiseHistory.filter(p => p.kept !== null);
const kept = resolved.filter(p => p.kept === true);
const promiseRate = resolved.length > 0 ? Math.round((kept.length / resolved.length) * 100) : 0;
```

Dynamic Encouragement:

- If sessions > 0: "You showed up X time(s). You kept Y promise(s)... Those are not the numbers of someone who is failing."
- If no sessions: "You are here right now, looking for help instead of giving up. That counts for something."

Key Details:

- Lazy-loaded: `fetchEvidence()` called only when entering evidence phase
- Fallback zeros on error (except streak from profile)
- Loading state with `ActivityIndicator`

4.3 Edit Profile

File: `app/edit-profile.tsx`

Purpose: Edit display name, main goal, and password.

What Changed: Replaced "In a real app" stub comments with real persistence.

Data Integration:

- **Initialization:** `profile?.display_name` for name, `getActiveGoal(userId)` for goal title/ID
- **Save Profile:** `useAuthStore.getState().updateProfile({ display_name }) + supabase.from('goals').update({ title }).eq('id', goalId)`
- **Change Password:** `supabase.auth.updateUser({ password: newPassword })`

Key Details:

- Email shown read-only (not editable)
- Password validation: both fields required, 8+ chars, must match
- No "Current Password" field (Supabase `updateUser` doesn't require it when authenticated)
- State: `displayName`, `mainGoal`, `goalId`, `isSaving`, `newPassword`, `confirmPassword`, `isChangingPassword`
- Success alerts with `router.back()` on dismiss

4.4 QuickActions - Promises Navigation Fix

File: `src/components/home/QuickActions.tsx`

What Changed: Added `onPromises` prop and "Promises" action button to QuickActions component.

Before: 3 actions (Start Session, Chat, Progress) — no way to reach Promises from home.

After: 4 actions with proper callbacks:

- Start Session → `[>]` icon, `bg-primary/10`
- Promises → `[*]` icon, `bg-secondary/10`
- Chat → `[C]` icon, `bg-accent/10`
- Progress → `[P]` icon, `bg-warning/10`

Home Integration: `app/(tabs)/index.tsx` passes `onPromises={handlePromises}` which calls `router.push('/promises')`.

These are **production infrastructure** items, not code gaps:

Item	Status	Dependency
RevenueCat integration (paywall purchase flow)	Placeholder	Requires RevenueCat SDK setup + App Store/Play Store products
Data export email delivery	Placeholder	Requires backend email service (e.g., Resend, SendGrid)
Push notifications (daily check-in reminder, timer, streak)	Not started	Requires expo-notifications setup + APNS/FCM config
Trial banner on home screen	Hardcoded hidden	Requires live subscription service check
Privacy toggles (anonymous sharing, analytics)	UI only	Requires analytics SDK integration

Generated March 15, 2026. All features verified via comprehensive code audit across 50+ files.